

Erros de execução em programas

Como captar a existência de erros a execução dos programas?

Semântica operacional estrutural (small-step):

Definir o sistema de regras das transição de forma a que comandos avaliados em estados que dêem origem a expressões erróneas conduzam a **configurações bloqueadas**.

Semântica natural (big-step):

Considerar a existência de um **estado de erro**, $\perp$, e considerar como estados finais possíveis os elementos do conjunto

$$\text{State}_{\perp} = \text{State} \cup \{\perp\}$$

Os comando só podem executar em estados pertencentes a **State**.

Uma Máquina Abstracta

Máquinas abstractas

- **Máquinas abstractas**
 - ▶ são **máquinas** porque permitem a execução passo a passo dos programas;
 - ▶ são **abstractas** porque omitem os detalhes das máquinas reais.
- Fornecem uma **linguagem intermédia** para a compilação, fazendo a ponte entre as linguagens de programação e as linguagens máquina reais.
 - ▶ As instruções de uma máquina abstracta devem ser talhadas de acordo com as características da linguagem de programação.
- É comum lidarem com conceitos como: **state, store, stack, registers**.
- **Emulador, interpretador, e máquina virtual** são termos alternativos para este conceito.
- Tornam a implementação das linguagens de programação mais portáteis e mais fáceis de manter.

Uma implementação correcta

- A especificação formal da semântica de uma linguagem de programação permite-nos argumentar acerca da correcção da sua implementação.
 - Vamos ilustrar isso definindo uma tradução da linguagem **While** para o assembly de uma máquina abstracta e **provar que a tradução é correcta**.
- Como?**
- ▶ Damos significado às instruções da máquina abstracta através de uma semântica operacional.
 - ▶ Definimos uma função de tradução que mapeia expressões e comandos da linguagem **While** em sequências de instruções da máquina abstracta.
 - ▶ Demonstramos que o significado do programa traduzido e do programa original coincidem.

A máquina abstracta **AM**

- A máquina abstracta **AM** tem configurações da forma $\langle c, e, s \rangle$ sendo
 - c a *sequência de instruções* (ou *código*) a ser executada
 - e a *stack* de avaliação das expressões aritméticas e booleanas (formalmente é uma lista de valores)
 - s o *storage* (ou *state*) (o estado das variáveis)
- As *instruções* de **AM** são dadas pela sintaxe abstracta:

$$\begin{aligned} inst &::= \text{PUSH-}n \mid \text{ADD} \mid \text{MULT} \mid \text{SUB} \\ &\mid \text{TRUE} \mid \text{FALSE} \mid \text{EQ} \mid \text{LE} \mid \text{AND} \mid \text{NEG} \\ &\mid \text{FETCH-}x \mid \text{STORE-}x \\ &\mid \text{NOOP} \mid \text{BRANCH}(c, c) \mid \text{LOOP}(c, c) \\ c &::= \varepsilon \mid inst:c \end{aligned}$$

Code é a categoria sintática das sequências de instruções.

Stack = $(\mathbf{Z} \cup \mathbf{T})^*$ **State** = $\mathbf{Var} \rightarrow \mathbf{Z}$

Semântica operacional de **AM**

A semântica do código da máquina **AM** é dado por um sistema de transição cujas configurações são

- $\langle c, e, s \rangle \in \mathbf{Code} \times \mathbf{Stack} \times \mathbf{State}$
- $\langle \epsilon, e, s \rangle$ é uma *configuração terminal* (ou *final*).

A relação de transição entre configurações $\langle c, e, s \rangle \triangleright \langle c', e', s' \rangle$ representa um passo de execução.

- ϵ representa a sequência vazia;
- $:$ a concatenação de sequências;
- o topo da stack é o lado esquerdo da sequência.

Semântica operacional de **AM**

$$\begin{aligned} \langle \text{PUSH-}n:c, e, s \rangle &\triangleright \langle c, \mathcal{N}[[n]]:e, s \rangle \\ \langle \text{ADD}:c, z_1:z_2:e, s \rangle &\triangleright \langle c, (z_1+z_2):e, s \rangle \quad \text{if } z_1, z_2 \in \mathbf{Z} \\ \langle \text{MULT}:c, z_1:z_2:e, s \rangle &\triangleright \langle c, (z_1 \star z_2):e, s \rangle \quad \text{if } z_1, z_2 \in \mathbf{Z} \\ \langle \text{SUB}:c, z_1:z_2:e, s \rangle &\triangleright \langle c, (z_1 - z_2):e, s \rangle \quad \text{if } z_1, z_2 \in \mathbf{Z} \\ \langle \text{TRUE}:c, e, s \rangle &\triangleright \langle c, \mathbf{tt}:e, s \rangle \\ \langle \text{FALSE}:c, e, s \rangle &\triangleright \langle c, \mathbf{ff}:e, s \rangle \\ \langle \text{EQ}:c, z_1:z_2:e, s \rangle &\triangleright \langle c, (z_1 = z_2):e, s \rangle \quad \text{if } z_1, z_2 \in \mathbf{Z} \\ \langle \text{LE}:c, z_1:z_2:e, s \rangle &\triangleright \langle c, (z_1 \leq z_2):e, s \rangle \quad \text{if } z_1, z_2 \in \mathbf{Z} \end{aligned}$$

Semântica operacional de **AM**

$$\begin{aligned} \langle \text{AND}:c, t_1:t_2:e, s \rangle &\triangleright \begin{cases} \langle c, \mathbf{tt} : e, s \rangle & \text{if } t_1 = \mathbf{tt} \text{ and } t_2 = \mathbf{tt} \\ \langle c, \mathbf{ff} : e, s \rangle & \text{if } t_1 = \mathbf{ff} \text{ or } t_2 = \mathbf{ff}, \text{ and } t_1, t_2 \in \mathbf{T} \end{cases} \\ \langle \text{NEG}:c, t:e, s \rangle &\triangleright \begin{cases} \langle c, \mathbf{ff} : e, s \rangle & \text{if } t = \mathbf{tt} \\ \langle c, \mathbf{tt} : e, s \rangle & \text{if } t = \mathbf{ff} \end{cases} \end{aligned}$$

Semântica operacional de **AM**

$$\begin{aligned}
 \langle \text{FETCH-}x:c, e, s \rangle &\triangleright \langle c, (s \ x):e, s \rangle \\
 \langle \text{STORE-}x:c, z:e, s \rangle &\triangleright \langle c, e, s[x \mapsto z] \rangle \quad \text{if } z \in \mathbf{Z} \\
 \langle \text{NOOP}:c, e, s \rangle &\triangleright \langle c, e, s \rangle \\
 \langle \text{BRANCH}(c_1, c_2):c, t:e, s \rangle &\triangleright \begin{cases} \langle c_1 : c, e, s \rangle & \text{if } t = \mathbf{tt} \\ \langle c_2 : c, e, s \rangle & \text{if } t = \mathbf{ff} \end{cases} \\
 \langle \text{LOOP}(c_1, c_2):c, e, s \rangle &\triangleright \langle c_1 : \text{BRANCH}(c_2 : \text{LOOP}(c_1, c_2), \text{NOOP}):c, e, s \rangle
 \end{aligned}$$

Sequência de computação

- Uma *sequência de computação* de uma sequência de instruções c num estado s pode ser uma de duas coisas:
 - ▶ uma sequência **finita** de configurações $\gamma_0, \gamma_1, \gamma_2, \dots, \gamma_k$ tais que $\gamma_0 = \langle c, \epsilon, s \rangle$, $\gamma_i \triangleright \gamma_{i+1}$ com $0 \leq i < k$, $k \geq 0$ e γ_k é uma configuração terminal ou bloqueada;
 - ▶ uma sequência **infinita** de configurações $\gamma_0, \gamma_1, \gamma_2, \dots$ tais que $\gamma_0 = \langle c, \epsilon, s \rangle$, $\gamma_i \triangleright \gamma_{i+1}$ para $0 \leq i$.
- As *configurações iniciais* têm sempre a stack vazia.
- Uma sequência de computação finita pode terminar numa configuração final ou bloqueada.

Semântica operacional de **AM**

Exemplos: Assumindo que $s \ x = 3$.

$$\begin{aligned}
 &\langle \text{PUSH-1:FETCH-}x:\text{ADD:STORE-}x, \epsilon, s \rangle \\
 &\triangleright \langle \text{FETCH-}x:\text{ADD:STORE-}x, \mathbf{1}, s \rangle \\
 &\triangleright \langle \text{ADD:STORE-}x, \mathbf{3:1}, s \rangle \\
 &\triangleright \langle \text{STORE-}x, \mathbf{4}, s \rangle \\
 &\triangleright \langle \epsilon, \epsilon, s[x \mapsto \mathbf{4}] \rangle \\
 \\
 &\langle \text{LOOP}(\text{TRUE}, \text{NOOP}), \epsilon, s \rangle \\
 &\triangleright \langle \text{TRUE:BRANCH}(\text{NOOP:LOOP}(\text{TRUE}, \text{NOOP}), \text{NOOP}), \epsilon, s \rangle \\
 &\triangleright \langle \text{BRANCH}(\text{NOOP:LOOP}(\text{TRUE}, \text{NOOP}), \text{NOOP}), \mathbf{tt}, s \rangle \\
 &\triangleright \langle \text{NOOP:LOOP}(\text{TRUE}, \text{NOOP}), \epsilon, s \rangle \\
 &\triangleright \langle \text{LOOP}(\text{TRUE}, \text{NOOP}), \epsilon, s \rangle \\
 &\triangleright \dots
 \end{aligned}$$

Semântica operacional de **AM**

Exercício: Indique a função é implementada pelo seguinte código:

```

PUSH-0:STORE-z:FETCH-x:STORE-r:
LOOP(FETCH-r:FETCH-y:LE,
    FETCH-y:FETCH-r:SUB:STORE-r:
    PUSH-1:FETCH-z:ADD:STORE-z)
    
```

Propriedade de AM

Lema

Se $\langle c_1, e_1, s \rangle \triangleright^k \langle c', e', s' \rangle$ então $\langle c_1 : c_2, e_1 : e_2, s \rangle \triangleright^k \langle c' : c_2, e' : e_2, s' \rangle$.

Lema

Se $\langle c_1 : c_2, e, s \rangle \triangleright^k \langle \epsilon, e'', s'' \rangle$ então $\langle c_1, e, s \rangle \triangleright^{k_1} \langle \epsilon, e', s' \rangle$ e $\langle c_2, e', s' \rangle \triangleright^{k_2} \langle \epsilon, e'', s'' \rangle$, para alguma configuração $\langle c_2, e', s' \rangle$ e números naturais k_1 e k_2 tais que $k = k_1 + k_2$.

Teorema

Para quaisquer $\gamma, \gamma', \gamma''$, se $\gamma \triangleright \gamma'$ e $\gamma \triangleright \gamma''$ então $\gamma' = \gamma''$.

A função de execução $\mathcal{M}$

O significado de uma sequência de instruções pode ser visto como uma **função parcial** de State para State.

Função semântica $\mathcal{M}$

$$\mathcal{M} : \text{Code} \rightarrow (\text{State} \rightarrow \text{State})$$

$$\mathcal{M}[\![c]\!] s = \begin{cases} s' & \text{se } \langle c, \epsilon, s \rangle \triangleright^* \langle \epsilon, e, s' \rangle \\ \text{undef} & \text{caso contrário} \end{cases}$$

A boa definição de $\mathcal{M}$ é uma consequência do determinismo da relação de transição.

Especificação da tradução

Como gerar código AM para um dado programa?

- A tradução de um programa **While** gera código **AM**.
- Precisamos de traduzir expressões e comandos.
- Cada tradução é especificada por uma função total.

Tradução de expressões aritméticas

$$\mathcal{CA} : \text{Aexp} \rightarrow \text{Code}$$

$$\begin{aligned} \mathcal{CA}[\![n]\!] &= \text{PUSH} - n \\ \mathcal{CA}[\![x]\!] &= \text{FETCH} - x \\ \mathcal{CA}[\![a_1 + a_2]\!] &= \mathcal{CA}[\![a_2]\!] : \mathcal{CA}[\![a_1]\!] : \text{ADD} \\ \mathcal{CA}[\![a_1 * a_2]\!] &= \mathcal{CA}[\![a_2]\!] : \mathcal{CA}[\![a_1]\!] : \text{MULT} \\ \mathcal{CA}[\![a_1 - a_2]\!] &= \mathcal{CA}[\![a_2]\!] : \mathcal{CA}[\![a_1]\!] : \text{SUB} \end{aligned}$$

Tradução de expressões booleanas

$CB : Bexp \rightarrow Code$

$$\begin{aligned} CB[\text{true}] &= \text{TRUE} \\ CB[\text{false}] &= \text{FALSE} \\ CB[a_1 = a_2] &= CA[a_2] : CA[a_1] : \text{EQ} \\ CB[a_1 \leq a_2] &= CA[a_2] : CA[a_1] : \text{LE} \\ CB[\neg b] &= CB[b] : \text{NEG} \\ CB[b_1 \wedge b_2] &= CB[b_2] : CB[a_1] : \text{AND} \end{aligned}$$

Tradução de comandos

$CS : Stm \rightarrow Code$

$$\begin{aligned} CS[x := a] &= CA[a] : \text{STORE} - x \\ CS[\text{skip}] &= \text{NOOP} \\ CS[C_1; C_2] &= CS[C_1] : CS[C_2] \\ CS[\text{if } b \text{ then } C_1 \text{ else } C_2] &= CB[b] : \text{BRANCH}(CS[C_1], CS[C_2]) \\ CS[\text{while } b \text{ do } C] &= \text{LOOP}(CB[b], CS[C]) \end{aligned}$$

A função de semântica $\mathcal{S}_{am}$

O significado de um programa **While** pode ser obtido, primeiro traduzindo-o para código **AM**, e depois executando o código gerado na máquina abstracta.

Função semântica $\mathcal{S}_{am}$

$$\mathcal{S}_{am} : Stm \rightarrow (\text{State} \hookrightarrow \text{State})$$

$$\mathcal{S}_{am}[C] = (\mathcal{M} \circ CS)[C]$$

Correcção da tradução

Lema

Para toda a expressão aritmética a e estado s ,

$$\langle CA[a], \epsilon, s \rangle \triangleright^* \langle \epsilon, \mathcal{A}[a] s, s \rangle$$

Além disso, todas as configurações intermédias desta sequência de computação têm uma stack não vazia.

Prova: Por indução estrutural em a .

Lema

Para toda a expressão booleana b e estado s ,

$$\langle CB[b], \epsilon, s \rangle \triangleright^* \langle \epsilon, \mathcal{B}[b] s, s \rangle$$

Além disso, todas as configurações intermédias desta sequência de computação têm uma stack não vazia.

Correcção da tradução

- Para formular a correcção da tradução de comandos podemos usar semântica natural ou semântica operacional estrutural.
- Vamos aqui ilustrar a prova seguindo a abordagem da semântica natural.
- Queremos provar que para qualquer programa C de **While**,

$$\mathcal{S}_{\text{ns}}[C] = \mathcal{S}_{\text{am}}[C]$$

Correcção da tradução

Lema

Para qualquer programa C e estados s e s' ,

$$\text{se } \langle C, s \rangle \rightarrow s' \text{ então } \langle \mathcal{CS}[C], \epsilon, s \rangle \triangleright^* \langle \epsilon, \epsilon, s' \rangle .$$

Prova: Por indução na estrutura da derivação de $\langle C, s \rangle \rightarrow s'$.

Lema

Para qualquer programa C e estados s e s' ,

$$\text{se } \langle \mathcal{CS}[C], \epsilon, s \rangle \triangleright^k \langle \epsilon, e, s' \rangle \text{ então } \langle C, s \rangle \rightarrow s' \text{ e } e = \epsilon .$$

Prova: Por indução no comprimento da sequência de computação.

Correcção da implementação

Teorema

Para qualquer programa C da linguagem **While**

$$\mathcal{S}_{\text{ns}}[C] = \mathcal{S}_{\text{am}}[C]$$

Prova: Consequência directa dos dois lemas anteriores.

É evidente que a implementação também é correcta em relação à semântica operacional estrutural. Isto é, para qualquer programa C da linguagem **While**

$$\mathcal{S}_{\text{sos}}[C] = \mathcal{S}_{\text{am}}[C]$$

Porquê?